DevOps Mini Project Report: Scientific Calculator

Author: Piyush Singh
Roll Number: MT2025088

1 Introduction

DevOps is a modern software development methodology that integrates development and operations to automate and improve the software delivery lifecycle. It focuses on continuous integration, continuous testing, continuous deployment, and monitoring of applications.

This project demonstrates a complete DevOps workflow by building a Scientific Calculator application and automating its build, testing, containerization, and deployment using Jenkins, Docker, Docker Hub, and Ansible.

2 What and Why of DevOps

2.1 What is DevOps?

DevOps is a combination of development (Dev) and operations (Ops). It aims to shorten the development lifecycle and deliver high-quality software continuously by improving collaboration between development and operations teams.

2.2 Why DevOps?

- **Speed:** Faster development and deployment cycles.
- **Rapid Delivery:** Continuous updates and improvements.
- **Reliability:** Automated testing ensures stability.
- **Scalability:** Infrastructure automation allows easy scaling.

3 System Architecture

The DevOps workflow implemented in this project follows an automated CI/CD pipeline where the source code is stored in GitHub, built and tested using Jenkins, containerized using Docker, stored in Docker Hub, and deployed using Ansible.

Developer
↓
GitHub Repository
↓
Jenkins Pipeline
↓
Run Tests
↓
Build Docker Image
↓
Push Image to Docker Hub
↓
Ansible Deployment
↓
Running Docker Container

Figure 1: DevOps CI/CD Pipeline Architecture

4 Tools Used

Tool	Category	Role in Project
Python	Development	Implements the scientific calculator functionality.
Git & GitHub	Version Control	Stores source code and tracks changes.
Jenkins	CI/CD	Automates build, test, and deployment pipeline.
Docker	Containerization	Packages the application and dependencies into containers.
Docker Hub	Registry	Stores Docker images for deployment.
Ansible	Deployment	Automates container deployment on the server.

5 Pipeline Stages Explanation

The CI/CD pipeline automates the lifecycle of the Scientific Calculator application.

5.1 Stage 1: Source Control Management

- Code is stored in a GitHub repository.
- Jenkins pulls the latest code whenever the pipeline runs.
- The repository is cloned into the Jenkins workspace.

5.2 Stage 2: Install Dependencies

Dependencies required for the application are installed.

```
pip3 install -r requirements.txt
```

5.3 Stage 3: Unit Testing

Automated tests are executed to verify the calculator functionality.

```
python3 -m unittest test_calculator.py
```

If tests fail, the pipeline stops.

5.4 Stage 4: Docker Login

Jenkins logs into Docker Hub using stored credentials.

- Credentials are securely stored in Jenkins.
- This step allows Jenkins to push Docker images.

5.5 Stage 5: Docker Build

Jenkins builds the Docker image using the Dockerfile.

```
docker build -t kabubabu/scientific-calculator:latest .
```

5.6 Stage 6: Docker Push

The Docker image is pushed to Docker Hub.

```
docker push kabubabu/scientific-calculator:latest
```

5.7 Stage 7: Ansible Deployment

Ansible deploys the Docker container automatically.

```
ansible-playbook -i inventory deploy.yml
```

This playbook:

- Pulls the latest Docker image
- Stops any existing container
- Runs a new container

6 Jenkins Pipeline Successful Build

The following screenshot shows the successful execution of the Jenkins pipeline.

Figure 2: Successful Jenkins Pipeline Execution

7 Running the Application Using Docker

Pull the Docker image from Docker Hub:

```
docker pull kabubabu/scientific-calculator:latest
```

Run the container:

```
docker run -it kabubabu/scientific-calculator:latest
```

The calculator application will start inside the container.

8 Scientific Calculator Application Screenshots

```
Scientific Calculator
1. Square Root
2. Factorial
3. Natural Log (ln)
4. Power (x^y)
5. Exit
Enter your choice: 4
Enter base: 4
Enter exponent: 2
Result: 16.0
```

```
Scientific Calculator
1. Square Root
2. Factorial
3. Natural Log (ln)
4. Power (x^y)
5. Exit
Enter your choice: 1
Enter number: 2
Result: 1.4142135623730951
```

```
Scientific Calculator
1. Square Root
2. Factorial
3. Natural Log (ln)
4. Power (x^y)
5. Exit
Enter your choice: 2
Enter number: 3
Result: 6
```

```
Scientific Calculator
1. Square Root
2. Factorial
3. Natural Log (ln)
4. Power (x^y)
5. Exit
Enter your choice: 3
Enter number: 4
Result: 1.3862943611198906
```

Figure 3: Scientific Calculator Operations

9 Project Links

- GitHub Repository: `https://github.com/Coder-Galaxy/scientific-calculator`
- Docker Hub Repository: `https://hub.docker.com/r/kabubabu/scientific-calculator`

10 Conclusion

This project demonstrates the implementation of a complete DevOps CI/CD pipeline for a Scientific Calculator application. By integrating GitHub, Jenkins, Docker, Docker Hub, and Ansible, the project automates the entire process of building, testing, containerizing, and deploying the application.

The DevOps pipeline improves development speed, reliability, and automation, ensuring faster and more efficient software delivery.